Hooks

A **Hook** is a special React function that lets functional components use React features.

useState()

useState()→ stores and changes data inside a component

Import the Hook in file where you use it.

```
import { useState, useEffect } from 'react';
```

Count increases when button is clicked code:

```
function App() {
  let count = 0;
  const increase = () => {
      count++;
      console.log(count);
  };
  return (
      <>
          <h1>{count}</h1>
          <button onClick={increase}>Increase</button>
      </>
  );
}
```

The following code wont work because:

But the UI **doesn't update**.

Why?

Because changing a normal JavaScript variable does **not tell React that the UI needs to be rendered again**.

ReactJS will only change the variable value in real time if they use Hooks.(useState).

Syntax

```
const [value, setValue] = useState(initialValue);
```

value is the variable name : count → current value

setValue is the function name that is used to change the state(value) of variable: setCount → change value

initialValue is the first value that is stored in the veariable: 0 → starting value

Example Code: To change the value of variable(name) when button is clicked.

```
import { useState } from "react";

function NameChange() {

    const [name, setName] = useState("Greshi");

    return (
        <div>
            <h1>Hello {name}</h1>

            <button onClick={() => setName("Rahul")}>
                Change Name
            </button>
        </div>
    );
}

export default NameChange;
```

Task: To make an increment and decrement counter using two buttons increase and decrease.

useEffect()

useEffect is used when you want to perform a side effect in a React component. A side effect means something that happens outside simply calculating JSX.

Common examples:

- API calls
- Fetching data
- Timers
- Updating document title
- Event listeners
- Subscriptions
- Running code when state changes

Syntax

```
// Reload = entire web page is loaded again
// Re-render = React runs the component again (not entire web page)
// Mount = component is added to the UI for the first time
// Unmount = component is removed from the UI

useEffect(() => {
•
•     // code to execute
•
• }, []);

//useEffect(function , dependency)
// dependencies:
// 1. No dependency array → runs after every render
// 2. Empty array [] → runs only on mount
// 3. [value] → runs on mount + whenever value changes
// 3. Mount + value ([value])
```

➔ No dependency array

useEffect(() => {

console.log("Effect");

});

Runs: After every render

➔ useEffect With Empty Dependency Array

useEffect(() => {

console.log("Effect");

}, []);

Runs: After initial render

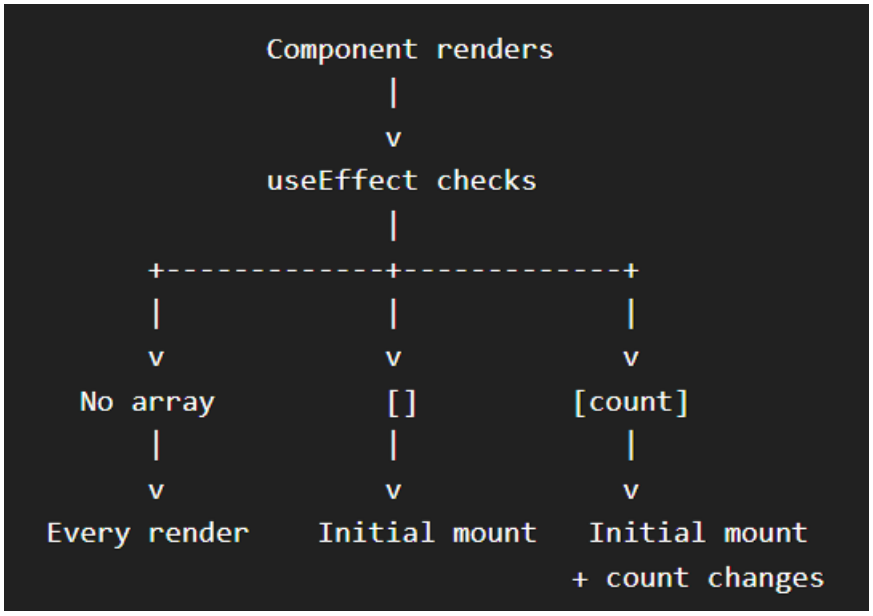
➔ useEffect with Dependency Array

useEffect(() => {

console.log("Effect");

}, [count]);

Runs: After initial render+When count changes



Code Example: Document title changes whenever count variable updates

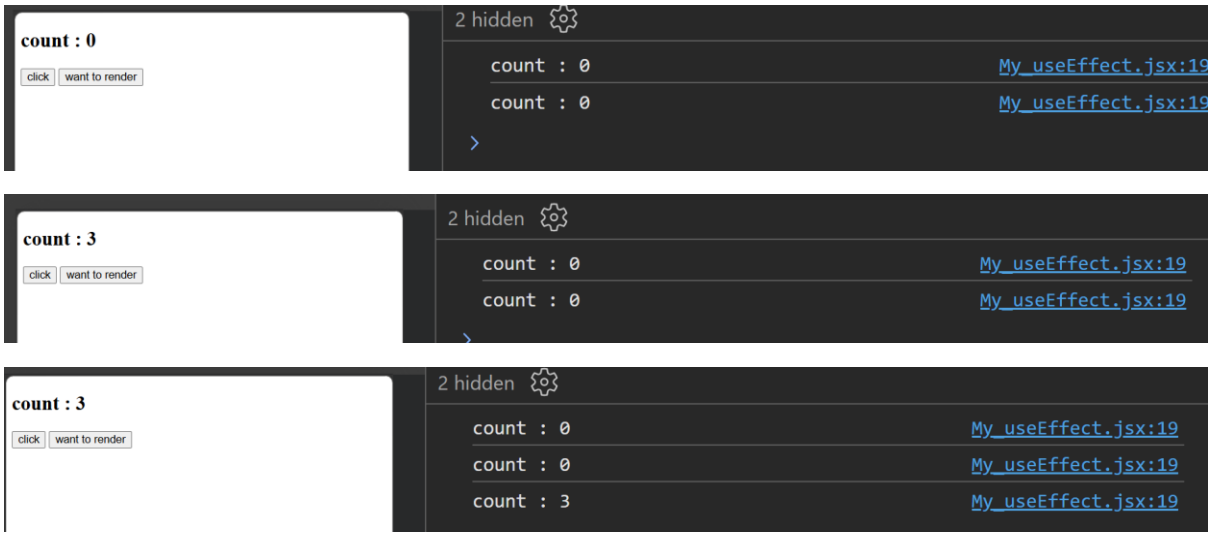
```jsx
import { useState, useEffect } from "react";

function DocTitleChange() {

    const [count, setCount] = useState(0);

    useEffect(() => {
        document.title = `Count: ${count}`;
    }, [count]);

    return (
        <div>

            <h1>Count: {count}</h1>

            <button onClick={() => setCount(count + 1)}>
                Increase
            </button>

        </div>
    );
}

export default DocTitleChange;
```

Task: Take two buttons:

Counter -> This updates the value of count variable (like 1,2,3++)

Want to render -> When clicked it prints the value of count variable in console. (Hint: It uses flag variable).

P.T.O for Mock Output



In simple words:

useState = "What data should I remember?"

useEffect = "What should I do after rendering or when this data changes?"